Conditional Statements in Java

Beginner-friendly notes | DSA in Java Series

1. What are Conditional Statements?

- Conditional statements let a program **make decisions**.
- The program checks a **condition** (a question with a true / false answer).
- Based on the result, Java decides **which block of code to run**.
- Without them, a program would run every line in order, every time.

Real-life example: *If it is raining, take an umbrella. Otherwise, leave it at home.*

Java has these conditional statements:

Statement	Used when
if	Run code only if a condition is true
if-else	Choose between two paths
else-if ladder	Choose between many paths
nested if	An if inside another if
switch	Match one variable against fixed values
ternary (? :)	Short one-line if-else

2. Operators Used in Conditions

A condition is built using **relational** and **logical** operators. The result is always a **boolean** (true or false).

Relational operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	true
!=	Not equal to	5 != 3	true
>	Greater than	4 > 9	false
<	Less than	4 < 9	true
>=	Greater than or equal	7 >= 7	true
<=	Less than or equal	8 <= 3	false

Logical operators

Operator	Name	Meaning
&&	AND	True only if **both** conditions are true
||	OR	True if **at least one** condition is true
!	NOT	Reverses the result (true becomes false)

Remember: use **==** to compare and **=** to assign. Writing **=** inside a condition is a very common beginner mistake.

3. The if Statement

Syntax

```
if (condition) {
    // code runs only when condition is true
}
```

- If the condition is **true**, the block runs.
- If the condition is **false**, the block is skipped.
- The condition must be written inside **( )** brackets.
- Use **{ }** curly braces to group the statements.

Example: check if a person can vote

```
int age = 20;

if (age >= 18) {
    System.out.println("You can vote");
}
```

Output: You can vote

4. The if-else Statement

Syntax

```
if (condition) {
    // runs when condition is true
} else {
    // runs when condition is false
}
```

- Exactly **one** of the two blocks always runs.
- **else** has no condition of its own.
- Use it when there are only two possible outcomes.

Example: even or odd

```
int num = 7;

if (num % 2 == 0) {
    System.out.println("Even number");
} else {
    System.out.println("Odd number");
}
```

Output: Odd number

% is the modulus operator. It gives the remainder after division. If the remainder when dividing by 2 is 0, the number is even.

5. The else-if Ladder

Syntax

```
if (condition1) {
    // block 1
} else if (condition2) {
    // block 2
} else if (condition3) {
    // block 3
} else {
    // runs if none of the above are true
}
```

- Conditions are checked **from top to bottom**.
- As soon as one condition is true, its block runs and the **rest are skipped**.
- The final **else** is optional. It acts as the default.
- Use it when there are three or more possible outcomes.

Example: grade calculator

```
int marks = 82;

if (marks >= 90) {
    System.out.println("Grade A");
} else if (marks >= 75) {
    System.out.println("Grade B");
} else if (marks >= 50) {
    System.out.println("Grade C");
} else {
    System.out.println("Fail");
}
```

Output: Grade B

6. Nested if

- A nested if is an **if statement inside another if**.
- The inner condition is checked only when the outer condition is true.
- Use it when a decision depends on another decision.

Example: largest of three numbers

```
int a = 10, b = 25, c = 15;

if (a > b) {
    if (a > c) {
        System.out.println("a is largest");
    } else {
        System.out.println("c is largest");
    }
} else {
    if (b > c) {
        System.out.println("b is largest");
    } else {
        System.out.println("c is largest");
    }
}
```

Output: b is largest

Same logic using logical operators (cleaner)

```
if (a >= b && a >= c) {
    System.out.println("a is largest");
} else if (b >= a && b >= c) {
    System.out.println("b is largest");
} else {
    System.out.println("c is largest");
}
```

7. The switch Statement

Syntax

```
switch (variable) {
    case value1:
        // code
        break;
    case value2:
        // code
        break;
    default:
        // runs if no case matches
}
```

- **switch** compares one variable with many fixed values.
- Each **case** is one possible value. After **case** comes a colon **:**
- **break** stops the switch after a match. Without it, execution **falls through** to the next case.
- **default** runs when no case matches. It is optional.

- Works with **int, char, String, byte, short** and enums. It does not work with float, double or long.
- Case values must be **constants**, and duplicate cases are not allowed.

Example: day of the week

```
int day = 3;

switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    default:
        System.out.println("Invalid day");
}
```

Output: Wednesday

Fall-through (what happens without break)

```
int n = 1;

switch (n) {
    case 1:
        System.out.println("One");
    case 2:
        System.out.println("Two");
    case 3:
        System.out.println("Three");
}
```

Output: One, Two, Three (all three lines print, because there is no break)

Example: simple calculator

```
char op = '+';
int x = 10, y = 5;

switch (op) {
    case '+': System.out.println(x + y); break;
    case '-': System.out.println(x - y); break;
    case '*': System.out.println(x * y); break;
    case '/': System.out.println(x / y); break;
    default:  System.out.println("Invalid operator");
}
```

Output: 15

8. Ternary Operator (? :)

- A short way to write a simple if-else in **one line**.
- It takes three parts, so it is called **ternary**.

Syntax

```
variable = (condition) ? valueIfTrue : valueIfFalse;
```

Example

```
int age = 16;
String result = (age >= 18) ? "Adult" : "Minor";
System.out.println(result);
```

Output: Minor

Use the ternary operator only for **simple** choices. For longer logic, a normal if-else is easier to read.

9. if-else vs switch

Point	if-else	switch
Checks	Any condition (ranges, &&, ||)	Only equality with fixed values
Data types	All types, including boolean and double	int, char, String, byte, short, enum
Best for	Ranges, e.g. marks >= 90	Menus, days, options, calculators
Readability	Gets long with many conditions	Cleaner for many fixed values
break needed	No	Yes (to avoid fall-through)

10. Common Mistakes to Avoid

- Using **=** instead of **==** in a condition.
- Putting a semicolon right after the if: **if (x > 5);** makes the block run every time.
- Forgetting **break** in a switch case.
- Skipping the curly braces and then adding a second line that is not part of the if.
- Writing **0 < x < 10**. Java needs **x > 0 && x < 10**.
- Comparing Strings with **==**. Use **.equals()** instead.

Wrong vs right

```
// Wrong
if (marks = 50) { }          // assignment, compile error

// Right
if (marks == 50) { }

// Comparing Strings
String s = "java";
if (s.equals("java")) { }    // correct way
```

11. Practice Programs

Try writing these yourself, from easy to slightly harder:

- Check if a number is positive, negative or zero

- Check if a number is even or odd
- Find the largest of two numbers
- Find the largest of three numbers
- Check if a year is a leap year
- Check if a character is a vowel or a consonant
- Check if a person is eligible to vote
- Grade calculator based on marks
- Simple calculator using switch
- Print the day name from a number (1 to 7)
- Check if a number is divisible by both 3 and 5
- Check if a triangle is valid from its three angles

12. Quick Summary

- **if**: run code only when the condition is true.
- **if-else**: choose between two options.
- **else-if ladder**: choose between many options. The first true condition wins.
- **nested if**: an if inside an if.
- **switch**: match one value against many fixed cases. Don't forget **break**.
- **Ternary**: a one-line if-else.
- Conditions always evaluate to **true or false**.
- Combine conditions with **&&**, **||** and **!**.

Practice is the key. Write each program yourself instead of copying it.